

Sistemi Operativi

Corso di Laurea in Informatica

Sapienza Università di Roma

a.a. 2026/2027

Marco Raoul Marini

marini@di.uniroma1.it

Gabriele Tolomei

tolomei@di.uniroma1.it

Lecture 1.3: C Programming I

Lecture Roadmap

"Understanding C means understanding the machine."

This lecture covers **8 major sections**:

1. **Variables, Scope & Lifetime** — naming memory
2. **Primitive Types & Sizes** — the type system
3. **Arrays (1D & 2D)** — sequences in memory
4. **Strings in C** — character arrays
5. **Pointers — Fundamentals** — addresses as values
6. **Pointer Arithmetic** — navigating memory
7. **Arrays–Pointers Relationship** — the deep connection
8. **Memory Safety** — writing correct C

By the end, you will think in terms of memory addresses.

| Why C Still Matters

C remains the language of **systems programming**:

- **Operating Systems** — Linux kernel, macOS XNU, Windows NT
- **Embedded Systems** — microcontrollers, firmware, RTOS
- **Compilers & Interpreters** — GCC, Clang, CPython runtime
- **Databases** — SQLite, PostgreSQL core
- **Performance-critical libraries** — libc, OpenSSL

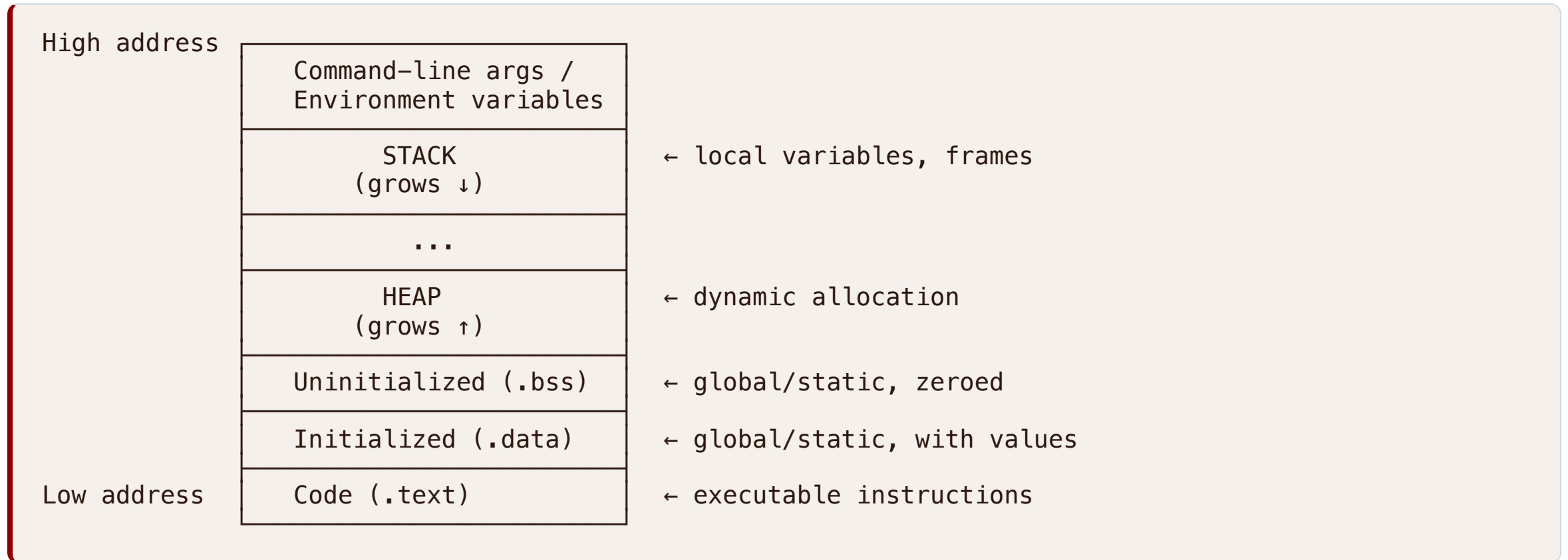
*C gives you **direct control** over memory — power that comes with responsibility.*

Key insight: Learning C teaches you how computers actually work.

§ The C Memory Model

| The C Memory Model (Preview)

Every running C program has a structured address space:



We will revisit this in Lecture 1.4 in full depth.

§ 1 — Variables, Scope & Lifetime

| What is a Variable?

A variable is a **named region of memory** that holds a value of a specific type.

Four attributes define a variable:

Attribute	Meaning	Example
Name	Identifier	counter
Type	What data it holds	int
Value	Current content	42
Address	Location in memory	0x7ffd3c20

```
int counter = 42;  
// Name:    counter  
// Type:    int (4 bytes)  
// Value:   42  
// Address: &counter → some stack address
```

Declaration vs. Definition

These terms are **not synonymous** in C:

Term	Meaning	Allocates memory?
Declaration	Announces existence, type	✗ No
Definition	Creates the variable	✓ Yes

```
// Declaration only (no memory allocated):  
extern int shared_counter;  
  
// Definition (memory allocated):  
int shared_counter = 0;  
  
// In most cases, declaration IS definition:  
int x;           // definition (uninitialized)  
int y = 10;      // definition with initialization
```

`extern` says: "this variable exists, but it's defined elsewhere."

Variable Initialization

Uninitialized variables contain garbage values — this is Undefined Behavior.

```
#include <stdio.h>

int main(void) {
    int x;           // ⚠ uninitialized: value is indeterminate
    printf("%d\n", x); // UB – could print anything, crash, etc.

    int y = 0;       // ✅ explicitly initialized
    int z = y + 1;    // ✅ initialized from expression
    return 0;
}
```

Rule of thumb: Always initialize variables at declaration.

Global and static variables are zero-initialized automatically by the C runtime.

| Scope: Block, File, and Function

Scope determines *where* in the code a variable is visible.

```
File scope (global)
int file_var = 10;
```

```
void foo() {
```

```
    Function/block scope
    int local = 5;
```

```
    if (local > 0) {
```

```
        Inner block scope
        int inner = 99;
```

```
    }
```

```
    // inner not visible here
```

Local Variables

Local variables live on the **stack** and are destroyed when their scope ends.

```
#include <stdio.h>

void print_sum(int a, int b) {
    int result = a + b;    // local to print_sum
    printf("Sum = %d\n", result);
} // result is destroyed here ← stack frame popped

int main(void) {
    print_sum(3, 7);
    // 'result' does not exist here
    return 0;
}
```

Lifetime: from point of declaration to end of enclosing `{ }`.

Never return a pointer to a local variable — classic bug (dangling pointer).

| Global Variables

Global variables are defined **outside any function** and live for the entire program duration.

```
#include <stdio.h>

int call_count = 0;    // global, in .data segment

void increment(void) {
    call_count++;      // accessible from any function
}

int main(void) {
    increment();
    increment();
    printf("Called %d times\n", call_count); // 2
    return 0;
}
```

Storage: `.data` (initialized) or `.bss` (zero-initialized)

Lifetime: entire program execution

Caution: globals make code harder to test and reason about.

Static Variables

The `static` keyword has **two different meanings** depending on context:

```
// (1) Static local: persists across calls, not re-initialized
void counter(void) {
    static int count = 0;    // initialized ONCE at program start
    count++;
    printf("count = %d\n", count);
}

// (2) Static global/function: limits visibility to THIS file
static int module_state = 0; // not visible outside this .c file
static void helper(void) { } // internal linkage

int main(void) {
    counter(); // count = 1
    counter(); // count = 2
    counter(); // count = 3
    return 0;
}
```

register and volatile

Two rarely-used but important qualifiers:

register — hint to compiler: store in CPU register

```
register int i;    // suggestion only; compiler may ignore it
// Cannot take address of register variable: &i is illegal
```

volatile — tells compiler: this value may change outside program control

```
volatile int sensor_value; // e.g., memory-mapped hardware register
// Prevents the compiler from caching/optimizing away reads
while (sensor_value == 0) { // always re-reads from memory
    // wait for hardware
}
```

volatile is critical in embedded systems and device drivers.

Scope Shadowing

An inner declaration can **shadow** an outer one — a common source of bugs:

```
#include <stdio.h>

int x = 100;    // global x

int main(void) {
    printf("global x = %d\n", x);    // 100

    int x = 200;    // shadows the global x
    printf("local x  = %d\n", x);    // 200

    {
        int x = 300;    // shadows the local x
        printf("inner x  = %d\n", x);    // 300
    }

    printf("local x  = %d\n", x);    // 200 again
    return 0;
}
```

Compilers can warn about this: use `-Wshadow` flag with GCC.

✓ Checkpoint 1 — Variables & Scope

Quick self-check:

1. What are the **four attributes** of a variable in C?
2. What is the difference between a **declaration** and a **definition**?
3. Where are **local variables** stored, and what is their lifetime?
4. What does `static` do when applied to a **local variable**?
5. What is **scope shadowing**, and why is it dangerous?

💬 *Discuss with a neighbor or write down your answers before continuing.*

§ 2 — Primitive Types & Sizes

| The C Type System

C is a **statically typed** language — every variable has a fixed type known at compile time.

Types serve three purposes:

1. **Size** — how many bytes to allocate
2. **Interpretation** — how to read/write those bytes
3. **Operations** — which operators are valid

C Types

- Integer types: `char`, `short`, `int`, `long`, `long long`
- Floating point: `float`, `double`, `long double`
- Void type: `void`
- Boolean: `_Bool` (C99), `bool` (`stdbool.h`)
- Derived types: arrays, pointers, structs, unions, enums
- Function types

char — The Byte Type

`char` is the **smallest addressable type**: exactly 1 byte (8 bits).

```
#include <stdio.h>

int main(void) {
    char c = 'A';           // character literal
    char n = 65;            // same as 'A' (ASCII 65)
    unsigned char byte = 255; // max unsigned byte value
    signed char s = -1;      // signed range: -128..127

    printf("char:          %c = %d\n", c, c);    // A = 65
    printf("unsigned char: %u\n", byte);         // 255
    printf("sizeof(char):  %zu\n", sizeof(char)); // 1
    return 0;
}
```

⚠ Whether plain `char` is signed or unsigned is **implementation-defined**. Always be explicit when it matters.

| int and Its Variants

The integer family, with **minimum guaranteed sizes**:

Type	Minimum bits	Typical (64-bit)	Signed range
char	8	1 byte	−128 .. 127
short	16	2 bytes	−32,768 .. 32,767
int	16	4 bytes	−2.1B .. 2.1B
long	32	8 bytes (Linux)	$\pm 9.2 \times 10^{18}$
long long	64	8 bytes	$\pm 9.2 \times 10^{18}$

```
short s = 32767;  
int i = 2147483647;  
long l = 9223372036854775807L;  
long long ll = 9223372036854775807LL;
```

| The `sizeof` Operator

`sizeof` returns the size **in bytes** of a type or expression — evaluated at compile time.

```
#include <stdio.h>

int main(void) {
    printf("char:      %zu bytes\n", sizeof(char));
    printf("short:     %zu bytes\n", sizeof(short));
    printf("int:        %zu bytes\n", sizeof(int));
    printf("long:        %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("float:       %zu bytes\n", sizeof(float));
    printf("double:      %zu bytes\n", sizeof(double));
    printf("pointer:     %zu bytes\n", sizeof(void *));
    return 0;
}

// On a 64-bit Linux system:
// char: 1, short: 2, int: 4, long: 8, long long: 8
// float: 4, double: 8, pointer: 8
```

Signedness: signed vs unsigned

All integer types can be **signed** (default) or **unsigned**:

```
signed int    si = -1;    // two's complement: 0xFFFFFFFF
unsigned int  ui = -1;    // wraps to 4294967295 (0xFFFFFFFF)

printf("%d\n",  si);      // -1
printf("%u\n",  ui);      // 4294967295
printf("%d\n",  ui);      // ⚠ UB: wrong format specifier
```

Two's complement for -1 (8-bit):

```
+1  = 0000 0001
    flip bits → 1111 1110
    add 1     → 1111 1111 = -1 in two's complement
```

Always use the correct **printf** format: **%d** for signed, **%u** for unsigned.

Integer Overflow

Signed overflow is **Undefined Behavior** in C — the compiler may assume it never happens.

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    int max = INT_MAX;           // 2,147,483,647
    printf("max = %d\n", max);
    printf("max + 1 = %d\n", max + 1); // ⚠ UB: signed overflow

    unsigned int umax = UINT_MAX; // 4,294,967,295
    printf("umax = %u\n", umax);
    printf("umax + 1 = %u\n", umax + 1); // ✅ wraps to 0 (defined)
    return 0;
}
```

***Unsigned overflow wraps** (modular arithmetic) — this is defined.*

***Signed overflow is UB** — compilers exploit this for optimization.*

| float and double

Floating-point types follow **IEEE 754**:

Type	Size	Precision	Range
float	4 bytes	~7 decimal digits	$\pm 3.4 \times 10^{38}$
double	8 bytes	~15 decimal digits	$\pm 1.8 \times 10^{308}$
long double	10–16 bytes	≥ 18 digits	platform-dependent

```
float f = 3.14f;           // 'f' suffix required for float literal
double d = 3.14159265358979;

printf("%.7f\n", f);       // 3.1400001 ← precision limit!
printf("%.15f\n", d);      // 3.141592653589790

// Dangerous comparison:
if (f == 3.14f) { }        // may fail due to rounding
// Better:
if (fabs(f - 3.14f) < 1e-6f) { }
```

`_Bool` / `bool` (C99)

C99 introduced a boolean type via `<stdbool.h>`:

```
#include <stdbool.h>
#include <stdio.h>

int main(void) {
    bool flag = true;
    bool off  = false;

    printf("true  = %d\n", true);    // 1
    printf("false = %d\n", false);   // 0
    printf("flag  = %d\n", flag);    // 1

    // Any non-zero value converts to true:
    bool b = 42;    // b = 1
    bool c = 0;     // c = 0

    if (flag) {
        puts("Flag is set");
    }
    return 0;
}
```

All Primitive Types – Summary Table

Type	Bytes	Typical Range
char	1	-128 .. 127
unsigned char	1	0 .. 255
short	2	-32,768 .. 32,767
unsigned short	2	0 .. 65,535
int	4	-2,147,483,648 .. 2,147,483,647
unsigned int	4	0 .. 4,294,967,295
long	4/8	platform-dependent
long long	8	$\pm 9.22 \times 10^{18}$
float	4	$\pm 3.4 \times 10^{38}$ (~7 digits)
double	8	$\pm 1.8 \times 10^{308}$ (~15 digits)
bool	1	0 or 1

Use `<stdint.h>` for **exact-width types**: `int8_t`, `uint32_t`, etc. — portable and unambiguous.

| Type Casting

C supports both **implicit** (automatic) and **explicit** (cast) conversions:

```
int    i = 300;
double d = i;           // implicit: int → double (safe, no data loss)
int    j = d;           // implicit: double → int (truncates! 300.9 → 300)

// Explicit cast using cast operator (type):
double pi = 3.14159;
int    ipi = (int)pi;    // 3 (truncates decimal)

int a = 7, b = 2;
double result = (double)a / b; // 3.5 (without cast: 3)
double wrong  = a / b;         // 3.0 (integer division first!)
```

Rule: When mixing types in arithmetic, make casts explicit to show intent.

typedef — Type Aliases

`typedef` creates an **alias** for an existing type, improving readability:

```
#include <stdint.h>
#include <stdio.h>

typedef unsigned char  byte_t;
typedef unsigned int   uint_t;
typedef int32_t        i32;

byte_t flags = 0xFF;
uint_t count = 1000;
i32     temperature = -15;

// Common pattern with structs:
typedef struct {
    int x;
    int y;
} Point;

Point p = {3, 7};
printf("(%d, %d)\n", p.x, p.y);
```

✓ Checkpoint 2 — Primitive Types & Sizes

Quick self-check:

1. What does `sizeof(int)` return on a typical 64-bit Linux system?
2. What is the difference between `signed` and `unsigned` overflow?
3. Why is comparing floating-point numbers with `==` dangerous?
4. What header provides `bool`, `true`, and `false` in C99?
5. Write a declaration of a variable that can hold the value `255` but **not** `256`.

💬 *Try writing the answers in C before advancing.*

§ 3 — Arrays

| What is an Array?

An **array** is a contiguous sequence of elements of the **same type**.

Key properties:

- Fixed size (determined at compile time for static arrays)
- Zero-indexed: first element is at index **0**
- All elements stored **adjacent** in memory

```
int arr[5] = {10, 20, 30, 40, 50};
```

Memory layout:

10	20	30	40	50
----	----	----	----	----

[0] [1] [2] [3] [4]
addr: base, base+4, base+8, ... (each int = 4 bytes)

1D Array Declaration

```
// Syntax: type name[size];

int  scores[5];           // 5 ints, uninitialized
char letters[26];        // 26 chars
double temps[12];        // 12 doubles

// With initialization:
int primes[5] = {2, 3, 5, 7, 11};

// Size inferred from initializer:
int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
// Compiler knows: 12 elements

// Partial initialization (rest zeroed):
int partial[10] = {1, 2, 3}; // [0]=1, [1]=2, [2]=3, rest=0
```

Array Initialization — All Forms

```
// Full explicit initialization:
int a[4] = {10, 20, 30, 40};

// Designated initializers (C99):
int b[5] = {[0] = 1, [3] = 9};    // b = {1, 0, 0, 9, 0}

// Zero-initialize entire array:
int c[100] = {0};    // all 100 elements set to 0
int d[100] = {};     // same (C99/C11)

// String array:
char greeting[] = "Hello";
// Equivalent to: {'H','e','l','l','o','\0'} - 6 elements!
```

The most common form is `= {0}` to safely zero an array at declaration.

Array Access & Indexing

```
#include <stdio.h>

int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};

    // Read:
    printf("First:  %d\n", arr[0]);    // 10
    printf("Last:   %d\n", arr[4]);    // 50

    // Write:
    arr[2] = 99;
    printf("Third:  %d\n", arr[2]);    // 99

    // ⚠ Out-of-bounds: UNDEFINED BEHAVIOR
    // arr[-1] = 5;    // Never do this!
    // arr[5] = 5;     // Index 5 doesn't exist in arr[5]

    return 0;
}
```

*C does **not** perform bounds checking. Out-of-bounds access is your responsibility.*

Array in Memory – Address Layout

Each element occupies `sizeof(type)` bytes, **contiguous**:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Address: 1000 1004 1008 1012 1016

10	20	30	40	50
----	----	----	----	----

[0] [1] [2] [3] [4]

`sizeof(arr) = 5 × 4 = 20 bytes`

Address of `arr[i]` = `base_address + i × sizeof(int)`
 = `1000 + i × 4`

*This formula is the foundation of **pointer arithmetic**.*

Iterating an Array

The canonical C pattern for array iteration:

```
#include <stdio.h>

#define N 5

int main(void) {
    int arr[N] = {3, 1, 4, 1, 5};
    int sum = 0;

    // Standard for-loop:
    for (int i = 0; i < N; i++) {
        sum += arr[i];
        printf("arr[%d] = %d\n", i, arr[i]);
    }
    printf("Sum = %d\n", sum);    // 14

    // Reverse iteration:
    for (int i = N - 1; i >= 0; i--) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Variable-Length Arrays (VLAs)

C99 allows array size to be determined at **runtime**:

```
#include <stdio.h>

void print_squares(int n) {
    int squares[n];    // VLA: size known only at runtime
    for (int i = 0; i < n; i++) {
        squares[i] = i * i;
    }
    for (int i = 0; i < n; i++) {
        printf("%d ", squares[i]);
    }
    printf("\n");
}

int main(void) {
    print_squares(6);    // 0 1 4 9 16 25
    return 0;
}
```

⚠ VLAs are **optional** in C11, not in C++. Avoid for large sizes (stack overflow risk). Prefer `malloc` for large dynamic arrays.

2D Arrays

A 2D array is an **array of arrays**:

```
// Declaration: type name[rows][cols];
int matrix[3][4];    // 3 rows, 4 columns = 12 ints

// Initialization:
int m[2][3] = {
    {1, 2, 3},    // row 0
    {4, 5, 6}    // row 1
};

// Access:
printf("%d\n", m[0][2]);    // 3  (row 0, col 2)
printf("%d\n", m[1][0]);    // 4  (row 1, col 0)

// Modify:
m[1][2] = 99;
```

2D Array Memory Layout

2D arrays are stored in **row-major order** in C:

```
int m[2][3] = {{1,2,3},{4,5,6}};
```

Logical view:

1	2	3
4	5	6

row 0

→

row 1

Memory (linear):

1	2	3	4	5	6
---	---	---	---	---	---

addr: +0 +4 +8 +12 +16 +20

$m[i][j]$ lives at: $\text{base} + (i \times 3 + j) \times \text{sizeof}(\text{int})$

$m[1][2] = \text{base} + (1 \times 3 + 2) \times 4 = \text{base} + 20$

*This is why accessing a 2D array row-by-row is **cache-friendly**.*

2D Array Access

```
#include <stdio.h>
#define ROWS 3
#define COLS 4

int main(void) {
    int m[ROWS][COLS] = {
        { 1,  2,  3,  4},
        { 5,  6,  7,  8},
        { 9, 10, 11, 12}
    };

    // Print the matrix:
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            printf("%3d ", m[i][j]);
        }
        printf("\n");
    }
    return 0;
}

// Output:
//   1  2  3  4
//   5  6  7  8
//   9 10 11 12
```

Passing Arrays to Functions

When you pass an array to a function, it **decays** to a pointer to its first element:

```
#include <stdio.h>

// These three declarations are EQUIVALENT:
void print_array(int arr[], int n);
void print_array(int *arr, int n);
void print_array(int arr[10], int n);    // size is ignored!

void print_array(int *arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main(void) {
    int data[5] = {1, 2, 3, 4, 5};
    print_array(data, 5);    // passes pointer to data[0]
    return 0;
}
```

The `sizeof` Gotcha with Array Parameters

```
#include <stdio.h>

void dangerous(int arr[]) {
    // arr is a POINTER here, not an array!
    printf("sizeof(arr) = %zu\n", sizeof(arr)); // 8 (pointer size!)
}

int main(void) {
    int data[10];
    printf("sizeof(data) = %zu\n", sizeof(data)); // 40 (10 × 4)
    dangerous(data); // prints 8, not 40!

    // Always pass the size explicitly:
    // dangerous(data, 10);
    return 0;
}
```

Golden rule: Always pass array size as a separate parameter.

`sizeof` on an array parameter gives you the **pointer size**, not the array size:

`sizeof(int)` // 4 bytes

`sizeof(int *)` // 8 bytes

✓ Checkpoint 3 — Arrays

Quick self-check:

1. What is the index of the **last element** of `int arr[7]` ?
2. How many bytes does `int m[3][4]` occupy?
3. What happens when you access `arr[n]` for an array of size `n` ?
4. What does an array **decay** to when passed to a function?
5. What does `sizeof(arr)` give you inside a function that received `arr` as a parameter?

💬 Sketch the memory layout of `int grid[2][2] = {{1,2},{3,4}}` .

§ 4 — Strings in C

Strings in C

C has **no built-in string type**. A string is a **char** array with a **null terminator** (`'\0'`, value 0):

```
char greeting[] = "Hello";
```

Memory layout:

'H'	'e'	'l'	'l'	'o'	'\0'
-----	-----	-----	-----	-----	------

0 1 2 3 4 5 ← indices

 ↑
 null terminator (REQUIRED)

```
sizeof(greeting) = 6    (includes '\0')
```

```
strlen(greeting) = 5    (excludes '\0')
```

String Literals

String literals are stored in **read-only memory** (`.rodata`):

```
// String literal – stored in .rodata, CANNOT be modified:
char *s1 = "Hello";
s1[0] = 'h';    // ⚠ UNDEFINED BEHAVIOR – segfault likely

// Array copy – modifiable, lives on stack:
char s2[] = "Hello";
s2[0] = 'h';    // ✅ OK – s2 is a local array copy

// Comparison:
printf("%p\n", (void *)s1); // pointer into .rodata
printf("%p\n", (void *)s2); // pointer into stack
```

Always use `char[]` (not `char*`) when you need a mutable string.

strcpy and strncpy

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char src[] = "Hello, world!";
    char dst1[50];
    char dst2[5];    // too small!

    // strcpy: NO bounds checking – DANGEROUS
    strcpy(dst1, src);    // ✓ dst1 is large enough
    // strcpy(dst2, src);    // ⚠ buffer overflow!

    // strncpy: safer – copies at most n bytes
    strncpy(dst2, src, sizeof(dst2) - 1);
    dst2[sizeof(dst2) - 1] = '\0';    // always null-terminate!

    printf("dst1: %s\n", dst1);
    printf("dst2: %s\n", dst2);    // "Hell"
    return 0;
}
```

strlen vs Array Size

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str[] = "Hello";

    printf("sizeof(str) = %zu\n", sizeof(str));    // 6 (includes '\0')
    printf("strlen(str) = %zu\n", strlen(str));    // 5 (excludes '\0')

    // Off-by-one error example:
    char buf[5];
    // strncpy(buf, str, sizeof(buf)); // copies "Hello" without '\0'!
    // buf[4] = '\0';                  // must add null manually

    // Safe pattern:
    strncpy(buf, str, sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';           // guarantee null termination
    printf("buf: %s\n", buf);              // "Hell"
    return 0;
}
```

strcmp — String Comparison

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char a[] = "apple";
    char b[] = "banana";
    char c[] = "apple";

    // WRONG: comparing pointers, not content
    // if (a == c) { ... } // always false (different addresses)

    // CORRECT: use strcmp
    printf("%d\n", strcmp(a, b)); // negative (a < b lexicographically)
    printf("%d\n", strcmp(b, a)); // positive
    printf("%d\n", strcmp(a, c)); // 0 (equal)

    if (strcmp(a, c) == 0) {
        puts("a and c are equal strings");
    }
    return 0;
}
```

sprintf and snprintf

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf[64];
    int day = 21, month = 5, year = 2025;

    // sprintf: like printf but into a string buffer
    sprintf(buf, "%04d-%02d-%02d", year, month, day);
    printf("Date: %s\n", buf);    // "2025-05-21"

    // snprintf: ALWAYS prefer this – limits output to n-1 chars
    snprintf(buf, sizeof(buf), "User %d logged in", 42);
    printf("%s\n", buf);

    // sprintf is DANGEROUS for variable-length input:
    // char small[10];
    // sprintf(small, "%s", "a very long string"); // overflow!

    return 0;
}
```

| String Pitfalls Summary

Common C string mistakes:

Pitfall	Cause	Fix
Buffer overflow	<code>strcpy</code> with no size check	Use <code>strncpy</code> + manual <code>'\0'</code>
Pointer comparison	<code>s1 == s2</code> for content	Use <code>strcmp</code>
Missing null terminator	Truncated copy	Always set <code>buf[n-1] = '\0'</code>
Modifying string literal	<code>char *s = "hello"; s[0]='x'</code>	Use <code>char s[] = "hello"</code>
Off-by-one	<code>strlen</code> vs <code>sizeof</code> confusion	Understand both carefully

Safe String Handling

```
#include <stdio.h>
#include <string.h>

// Safe copy function:
void safe_copy(char *dst, const char *src, size_t dst_size) {
    if (!dst || !src || dst_size == 0) return;
    strncpy(dst, src, dst_size - 1);
    dst[dst_size - 1] = '\0';
}

int main(void) {
    char buffer[16];
    safe_copy(buffer, "This is a very long string", sizeof(buffer));
    printf("%s\n", buffer);    // 'This is a very '

    // Always check return values and sizes:
    int written = snprintf(buffer, sizeof(buffer), "Item %d", 42);
    if (written >= (int)sizeof(buffer)) {
        fprintf(stderr, "Output was truncated!\n");
    }
    return 0;
}
```

✓ Checkpoint 4 — Strings

Quick self-check:

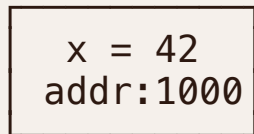
1. What character marks the **end** of a C string?
2. What is the difference between `sizeof("hello")` and `strlen("hello")` ?
3. Why is `if (str1 == str2)` **wrong** for comparing string contents?
4. Why is `strcpy` considered unsafe? What should you use instead?
5. Declare a **modifiable** C string containing `"world"` .

§ 5 — Pointers: Fundamentals

| What is a Pointer?

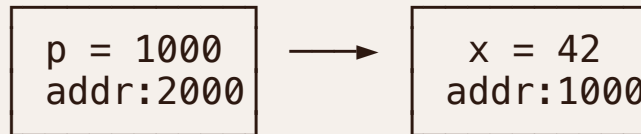
A **pointer** is a variable whose value is a **memory address**.

Regular variable:



"x stores 42"

Pointer variable:



"p stores the address of x"

Pointers enable:

- **Passing by reference** — modify variables in calling function
- **Dynamic memory** — work with heap-allocated data
- **Data structures** — linked lists, trees, graphs
- **Efficient arrays** — navigate large data without copying

Pointer Declaration

The `*` in a declaration means "this is a pointer to":

```
int    *pi;      // pointer to int
char   *pc;      // pointer to char
double *pd;      // pointer to double
int    **ppi;    // pointer to pointer to int
void   *pv;      // pointer to void (generic)

// ⚠ Declaration style matters:
int* a, b;      // a is int*, b is plain int ← common mistake!
int *a, *b;     // BOTH are int*           ← clearer style
```

Best practice: Put the `*` next to the variable name, not the type.

The Address-of Operator &

& applied to a variable gives you its **address**:

```
#include <stdio.h>

int main(void) {
    int x = 42;
    int *p;

    p = &x;    // p now holds the address of x

    printf("x      = %d\n", x);        // 42
    printf("&x     = %p\n", (void*)&x); // e.g. 0x7ffd1234
    printf("p      = %p\n", (void*)p); // same address
    printf("p==&x : %d\n", p == &x);  // 1 (true)

    return 0;
}
```

`%p` is the format specifier for printing pointers. Cast to `void*` for portability.

The Dereference Operator *

`*p` gives you the **value stored at** the address `p` points to:

```
#include <stdio.h>

int main(void) {
    int x = 42;
    int *p = &x;

    // Read through pointer (dereference):
    printf("*p = %d\n", *p);    // 42

    // Write through pointer:
    *p = 100;
    printf("x  = %d\n", x);    // 100 ← x was changed!
    printf("*p = %d\n", *p);    // 100

    return 0;
}
```

`*p = 100` modifies `x` through the pointer — this is **pass-by-reference** in C.

Pointer Diagram — Box and Arrow

Code:

```
int x = 42;  
int y = 7;  
int *p = &x;
```

Memory:

Variable	Address	Value
x	0x1000	[42]
y	0x1004	[7]
p	0x2000	[0x1000]



p → x

After: p = &y;

p 0x2000 [0x1004] → y

After: *p = 99;

y 0x1004 [99] (y changed via p)

Pointer Types Matter

The type of a pointer controls **how many bytes** are read/written on dereference:

```
#include <stdio.h>

int main(void) {
    int    n = 0x12345678;
    int    *pi = &n;
    char   *pc = (char *)&n;    // point to same address, different type

    printf("*pi    = 0x%08X\n", *pi);    // reads 4 bytes: 0x12345678
    printf("*pc    = 0x%02X\n", (unsigned char)*pc); // reads 1 byte

    // On little-endian: first byte is 0x78 (least significant)
    return 0;
}
```

Strict aliasing: You should not access the same memory through pointers of incompatible types — it's UB.
Exception: `char*` can alias anything.

NULL Pointer

NULL is a pointer that **points to nothing** (address 0):

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = NULL;    // p is explicitly "not pointing anywhere"

    // ALWAYS check before dereferencing:
    if (p != NULL) {
        printf("%d\n", *p);
    } else {
        printf("p is NULL, cannot dereference\n");
    }

    // Dereferencing NULL = SEGFAULT:
    // *p = 42;    // ⚠ crash: segmentation fault

    return 0;
}
```

Initialize pointers to **NULL** when you don't have an address yet. Check before use.

void * Pointers

`void *` is a **generic pointer** — it can hold any address but cannot be directly dereferenced:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int x = 42;
    void *vp = &x;          // any pointer can be assigned to void*

    // Can't dereference directly:
    // printf("%d\n", *vp);    // ✗ error

    // Must cast first:
    printf("%d\n", *(int *)vp); // ✓

    // malloc returns void*:
    void *buf = malloc(100);
    int *arr = (int *)buf;      // cast to use as int array
    free(buf);
    return 0;
}
```

Pointers to Pointers

A pointer to a pointer stores the address of another pointer:

```
#include <stdio.h>

int main(void) {
    int    x    = 42;
    int    *p    = &x;    // p → x
    int    **pp  = &p;    // pp → p → x

    printf("x    = %d\n",  x);    // 42
    printf("*p    = %d\n",  *p);    // 42
    printf("**pp= %d\n",  **pp);    // 42

    **pp = 99;    // modify x through pp → p → x
    printf("x    = %d\n",  x);    // 99

    return 0;
}
```

Used for: dynamic arrays of strings (`char **argv`), modifying pointer parameters, 2D dynamic arrays.

const and Pointers

const can qualify the **pointer**, the **pointed-to data**, or **both**:

```
int x = 10;
int y = 20;

const int *p1 = &x;    // pointer to const int: can change p1, not *p1
// *p1 = 99;           // ✗ error: data is read-only
p1 = &y;                // ✓ OK: can reassign pointer

int *const p2 = &x;    // const pointer to int: can change *p2, not p2
*p2 = 99;              // ✓ OK
// p2 = &y;            // ✗ error: pointer is read-only

const int *const p3 = &x; // const pointer to const int: neither writable
```

Use `const int *` for function parameters you promise not to modify.

✓ Checkpoint 5 — Pointer Basics

Quick self-check:

1. What is the difference between `int *p` and `*p`?
2. What does `&x` give you? What type does it have if `x` is `int`?
3. If `p = &x` and `*p = 5`, what is the value of `x`?
4. Why should you always check for `NULL` before dereferencing?
5. What is the difference between `const int *p` and `int * const p`?

§ 6 — Pointer Arithmetic & Arrays

Pointer Arithmetic Basics

When you add/subtract integers to/from a pointer, the pointer moves by **multiples of the element size**:

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;    // p points to arr[0]

// p    points to arr[0] = 10  (address: e.g. 1000)
// p+1  points to arr[1] = 20  (address: 1004)
// p+2  points to arr[2] = 30  (address: 1008)

printf("*p      = %d\n", *p);    // 10
printf("*(p+1) = %d\n", *(p+1)); // 20
printf("*(p+4) = %d\n", *(p+4)); // 50

// Conceptually, p + n advances by n * sizeof(*p) bytes.
// For int*: p+1 advances by 4 bytes as sizeof(*int) = 4 bytes.
```

Incrementing a Pointer

```
#include <stdio.h>

int main(void) {
    double temps[4] = {22.5, 18.3, 25.0, 30.1};
    double *p = temps;

    while (p < temps + 4) {          // p < one-past-end
        printf("%.1f\n", *p);
        p++;      // advance by sizeof(double) = 8 bytes
    }

    // Also valid:
    p = temps;
    p += 2;          // skip 2 elements
    printf("%.1f\n", *p); // 25.0 (index 2)

    return 0;
}
```

`p++` is equivalent to `p = p + 1` which is `p = (char*)p + sizeof(*p)`.

Pointer Subtraction

Subtracting two pointers of the **same type** gives the **number of elements** between them:

```
#include <stdio.h>
#include <stddef.h>    // ptrdiff_t

int main(void) {
    int arr[6] = {1, 2, 3, 4, 5, 6};
    int *first = &arr[1];
    int *last  = &arr[4];

    ptrdiff_t diff = last - first;    // 3 (not 12 bytes – elements!)
    printf("Elements between: %td\n", diff);    // 3

    // Finding the index of an element:
    int *found = &arr[3];
    ptrdiff_t index = found - arr;    // 3
    printf("Index: %td\n", index);

    return 0;
}
```

Pointer Comparison

Pointers within the **same array** can be compared with relational operators:

```
#include <stdio.h>

int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;
    int *q = &arr[3];

    printf("p < q: %d\n", p < q);    // 1 (true: arr+0 < arr+3)
    printf("p > q: %d\n", p > q);    // 0 (false)
    printf("p == q: %d\n", p == q);  // 0

    // Walk to the end:
    int *end = arr + 5;    // one past last element (valid pointer, don't deref)
    for (int *cur = arr; cur < end; cur++) {
        printf("%d ", *cur);
    }
    return 0;
}
```

Arrays Decay to Pointers

When an array is used in most expressions, it **decays** to a pointer to its first element:

Decay rule:

```
int arr[5] = {1, 2, 3, 4, 5};
```

<code>arr</code>	<code>→ &arr[0]</code>	<code>(type: int *)</code>
<code>arr + i</code>	<code>→ &arr[i]</code>	<code>(type: int *)</code>
<code>*arr</code>	<code>→ arr[0]</code>	<code>(value: 1)</code>

What does NOT decay:

<code>sizeof(arr)</code>	<code>→ 20</code>	<code>(array size, not pointer size)</code>
<code>&arr</code>	<code>→</code>	<code>pointer to the whole array, type: int (*)[5]</code>
<code>arr</code> in declaration context		

*This is why arrays and pointers are so closely related in C — but they are **not** the same.*

`arr[i]` is `*(arr + i)`

This is the formal definition of array indexing in C:

```
int arr[5] = {10, 20, 30, 40, 50};

// These are all IDENTICAL:
arr[2]           // standard notation
*(arr + 2)       // pointer arithmetic
*(2 + arr)       // commutative!
2[arr]           // weird but VALID in C (rarely used)

// Proof that 2[arr] works (for fun):
printf("%d\n", arr[2]);    // 30
printf("%d\n", *(arr + 2)); // 30
printf("%d\n", 2[arr]);    // 30 ← same!
```

`E1[E2]` is defined as `*((E1) + (E2))` by the C standard.

Iterating an Array with a Pointer

```
#include <stdio.h>

int sum_array(const int *arr, int n) {
    const int *end = arr + n;
    int total = 0;
    while (arr < end) {
        total += *arr;
        arr++;           // advance to next element
    }
    return total;
}

int main(void) {
    int data[6] = {3, 1, 4, 1, 5, 9};
    printf("Sum = %d\n", sum_array(data, 6));    // 23
    return 0;
}
```

Pointer iteration is equivalent to index-based — the compiler often generates the same code.

Pointer to Array vs Pointer to First Element

A subtle but important distinction:

```
int arr[5] = {1, 2, 3, 4, 5};

int *p = arr;           // pointer to int (first element)
int (*pa)[5] = &arr;    // pointer to int[5] (whole array)

// p has type int*      : p+1 advances by 4 bytes
// pa has type int(*)[5]: pa+1 advances by 20 bytes (whole array)

printf("sizeof(*p) = %zu\n", sizeof(*p));    // 4
printf("sizeof(*pa) = %zu\n", sizeof(*pa));  // 20

// Dereferencing pa gives the array:
printf("(*pa)[2] = %d\n", (*pa)[2]);        // 3
```

Multidimensional Arrays & Pointers

Pointer arithmetic with 2D arrays:

```
int m[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};

// m decays to pointer to first row: int (*)[4]
int (*row_ptr)[4] = m;

// Iterate rows:
for (int i = 0; i < 3; i++) {
    printf("Row %d: ", i);
    for (int j = 0; j < 4; j++) {
        printf("%d ", row_ptr[i][j]);    // same as m[i][j]
    }
    printf("\n");
}

// row_ptr + 1 advances by sizeof(int[4]) = 16 bytes
```

Function Pointers (Preview)

Pointers can also point to **functions**:

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int main(void) {
    // Declare a pointer to a function (int, int) → int
    int (*op)(int, int);

    op = add;
    printf("add(3,4) = %d\n", op(3, 4));    // 7

    op = mul;
    printf("mul(3,4) = %d\n", op(3, 4));    // 12

    return 0;
}
```

Function pointers enable **callbacks**, **dispatch tables**, and **polymorphism** in C. We'll explore them further in later lectures.

✓ Checkpoint 6 — Pointer Arithmetic

Quick self-check:

1. If `p` is an `int*` pointing to `arr[0]`, what does `p + 3` point to?
2. What is `arr[i]` equivalent to in pointer notation?
3. What is the result of `&arr[3] - &arr[1]` ?
4. What is the difference between `int *p = arr` and `int (*p)[5] = &arr` ?
5. When does an array **not** decay to a pointer?

§ 7 — Memory Safety

| Why Memory Safety Matters

Memory errors are among the most **dangerous and costly** bugs in software:

- ~**70%** of Microsoft CVEs are memory safety issues
- Buffer overflows enabled legendary exploits (Morris Worm 1988, Heartbleed 2014)
- Undefined Behavior can be silently exploited by compilers

Common memory bugs in C:

- | | |
|------------------------|--------------------------|
| 1. Dangling pointers | (use after free/scope) |
| 2. Wild pointers | (uninitialized pointer) |
| 3. Buffer overflow | (write past array end) |
| 4. Off-by-one errors | (fence-post errors) |
| 5. Uninitialized reads | (garbage values) |
| 6. Memory leaks | (covered in Lecture 1.4) |
| 7. Double free | (covered in Lecture 1.4) |

Dangling Pointers

A **dangling pointer** points to memory that is no longer valid:

```
#include <stdio.h>

int *get_local(void) {
    int x = 42;
    return &x;    // ⚠ returning address of local variable!
}                // x is destroyed here – stack frame popped

int main(void) {
    int *p = get_local();
    printf("%d\n", *p);    // ⚠ UB: p is now dangling!
                          // may print 42, may crash, may print garbage

    // Also dangling after free:
    int *q = malloc(sizeof(int));
    free(q);
    // *q = 5;    // ⚠ use after free – UB
    q = NULL;    // ✅ good practice: null out after free
    return 0;
}
```

Wild Pointers

A **wild pointer** is a pointer that has **never been initialized**:

```
#include <stdio.h>

int main(void) {
    int *p;           // ⚠ wild pointer: contains garbage address
    // *p = 42;        // ⚠ UB: writing to unknown memory location

    // Fix: always initialize
    int x = 10;
    int *safe = &x;    // ✅ points to valid memory
    *safe = 42;

    // Or initialize to NULL:
    int *null_p = NULL;
    if (null_p) {      // always check before use
        *null_p = 42;
    }

    return 0;
}
```

Buffer Overflow

A buffer overflow occurs when you **write past the end** of an array:

```
#include <string.h>
#include <stdio.h>

void vulnerable(const char *input) {
    char buf[8];
    strcpy(buf, input); // ⚠ NO size check!
    printf("buf = %s\n", buf);
}

int main(void) {
    vulnerable("OK"); // ✅ safe
    vulnerable("This is too long"); // ⚠ OVERFLOW!
    // Overwrites: return address, other stack variables...
    // Can lead to crash or arbitrary code execution
    return 0;
}
```

Buffer Overflow

A buffer overflow occurs when you **write past the end** of an array:

Stack before overflow:

buf[8]
saved EBP
return addr

←write

Stack after overflow:

"This is "
corrupted!
corrupted!

(16 bytes!)

← exploit

Off-by-One Errors

The most common array bug — accessing one element beyond the valid range:

```
#include <stdio.h>

int main(void) {
    int arr[5] = {1, 2, 3, 4, 5};

    // ⚠ Off-by-one: i <= 5 should be i < 5
    for (int i = 0; i <= 5; i++) {
        printf("%d\n", arr[i]);    // arr[5] is OUT OF BOUNDS
    }

    // ✅ Correct:
    for (int i = 0; i < 5; i++) {
        printf("%d\n", arr[i]);
    }

    // ⚠ String example:
    char s[] = "hello";
    // s[strlen(s)] = 'X';    // replaces '\0' – destroys string termination
    return 0;
}
```

Uninitialized Variables

Reading uninitialized variables is **Undefined Behavior**:

```
#include <stdio.h>

int main(void) {
    int x;           // uninitialized – contains whatever was on stack
    int arr[5];      // uninitialized – all 5 elements contain garbage

    if (x > 0) {     // ⚠ UB: reading uninitialized x
        // compiler may optimize this branch away!
    }

    // ✅ Always initialize:
    int y = 0;
    int data[5] = {0}; // zero-initializes all elements

    return 0;
}
```

Compilers with `-Wall -Wuninitialized` will warn about these. **Enable warnings.**

Aliasing Issues

Aliasing occurs when two pointers point to the **same memory**:

```
#include <stdio.h>

void add_to_self(int *a, int *b) {
    *a += *b;    // if a == b, this doubles *a
}

int main(void) {
    int x = 5;
    add_to_self(&x, &x);    // both point to same variable!
    printf("%d\n", x);      // 10 (not 5+5=10... but potentially ambiguous)

    // In optimized code, aliasing can cause subtle bugs
    // Use 'restrict' keyword to tell compiler pointers don't alias:
    // void safe_add(int * restrict a, const int * restrict b);

    return 0;
}
```

| Tools: Valgrind & AddressSanitizer

Two essential tools for finding memory errors:

Valgrind (memcheck):

```
gcc -g -O0 program.c -o program
valgrind --leak-check=full ./program
# Detects: memory leaks, invalid reads/writes, use of uninit memory
```

AddressSanitizer (ASan):

```
gcc -g -fsanitize=address,undefined program.c -o program
./program
# Much faster than Valgrind, catches: buffer overflow, use-after-free,
# stack overflows, double-free
```

*Both tools require **debug builds** (`-g` , no `-O2`). Use them routinely during development.*

Defensive Coding Patterns

```
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 1. Assert preconditions:
void process(int *arr, int n) {
    assert(arr != NULL);
    assert(n > 0);
    // ...
}

// 2. Check malloc return:
int *buf = malloc(n * sizeof(int));
if (!buf) {
    fprintf(stderr, "malloc failed\n");
    exit(EXIT_FAILURE);
}

// 3. Null pointers after free:
free(buf);
buf = NULL;

// 4. Use safe string functions:
strncpy(dst, src, sizeof(dst) - 1);
dst[sizeof(dst) - 1] = '\0';
```

✓ Checkpoint 7 — Memory Safety

Quick self-check:

1. What is a **dangling pointer**? Give two scenarios where one is created.
2. What is the difference between a **wild pointer** and a **NULL pointer**?
3. What command-line flag catches many memory errors at runtime?
4. Why is off-by-one in array bounds so dangerous?
5. What should you always do to a pointer **after** calling `free()` ?

| § Summary: Pointers

Key pointer takeaways:

- A pointer stores a **memory address**; `&x` gives the address of `x`
- `*p` dereferences — reads or writes the value at the address
- Pointer arithmetic scales by **element size**, not bytes
- `arr[i]` and `*(arr + i)` are **identical** by C definition
- Arrays **decay** to a pointer to their first element in most contexts
- `NULL` means "no valid address" — always check before dereferencing
- `const int *p` protects the data; `int * const p` protects the pointer
- Pointers enable pass-by-reference and efficient array traversal

| § Summary: Types & Arrays

Key types & arrays takeaways:

- Every variable has a **type**, **size**, **value**, and **address**
- Use `sizeof` to get portable sizes; use `<stdint.h>` for exact widths
- **Signed overflow** is UB; **unsigned overflow** wraps (defined)
- Floating-point comparison with `==` is dangerous — use epsilon
- Arrays are **contiguous** in memory; 2D arrays use row-major order
- Arrays do **not** carry their size when passed to functions
- C strings require a `'\0'` terminator — `sizeof` $\neq$ `strlen`
- Always pass array size as a separate parameter

Summary

Section	Core Concept
Variables & Scope	Name + type + value + address; scope controls visibility
Primitive Types	Fixed sizes; signed vs unsigned; IEEE 754 floats
Arrays	Contiguous memory; zero-indexed; decay to pointer
Strings	<code>char[]</code> + <code>'\0'</code> ; <code>string.h</code> functions; always check bounds
Pointer Basics	Address-valued variable; <code>&</code> , <code>*</code> , <code>NULL</code> , <code>const</code>
Pointer Arithmetic	Scales by element size; <code>arr[i] ≡ *(arr+i)</code>
Memory Safety	Dangling, wild, overflow, off-by-one; use ASan/Valgrind

Next lecture: How memory is actually managed — stack frames, heap allocation, the full compilation pipeline.

Further Reading & Resources

Resource	Focus
K&R — <i>The C Programming Language</i> (Kernighan & Ritchie)	Definitive reference
cppreference.com	C standard library, types, behavior
Beej's Guide to C (online, free)	Practical, approachable
GCC manual	<code>-Wall</code> , <code>-Wextra</code> , <code>-fsanitize</code> flags
Valgrind — valgrind.org	Memory error detection
CERT C Coding Standard	Security-focused C guidelines

```
# Recommended compilation flags for this course:  
gcc -std=c11 -Wall -Wextra -Wconversion -Wshadow \  
    -fsanitize=address,undefined -g \  
    -o program program.c
```

Next Lecture

Lecture 1.4: C Programming — Memory, Compiler, Linker, Assembler

We will explore:

1. **Stack frames** — how function calls work at the hardware level
2. **Heap allocation** — `malloc`, `calloc`, `realloc`, `free`
3. **Memory bugs** — leaks, double-free, dangling pointers in depth
4. **Compilation pipeline** — from `.c` to executable
 - Preprocessing → Compilation → Assembly → Linking
5. **Object files** — what's inside a `.o` file
6. **Static vs dynamic linking** — `.a` vs `.so` / `.dll`

Bring your curiosity about what the compiler actually does with your code.

| Lecture 1.3 Complete

Recommended Reading

- **K&R** — *The C Programming Language* (Kernighan & Ritchie)
- **cppreference.com** — C standard library, types, behavior
- **Beej's Guide to C** (online, free) — Practical, approachable
- **GCC manual** — `-Wall`, `-Wextra`, `-fsanitize` flags
- **Valgrind** — valgrind.org — Memory error detection
- **CERT C Coding Standard** — Security-focused C guidelines

Contact & Q&A

- **Lecturer:** Gabriele Tolomei
- **Email:** tolomei@di.uniroma1.it
- **Office Hours:** Drop me a message, and we'll arrange a time to meet, either in person or online! 😊

Thank you for your attention!